

ELI V2.0

What ELI Is & What ELI Can Do

A strictly local, private AI assistant and cognitive runtime — 208 capabilities behind one typed or spoken interface, running entirely on your own hardware. This is the honest tour: the idea behind it, the machinery under it, and every single thing it can actually do.

Edition: v2.0.16 · **Audience:** everyone — the curious and the technical · **Author:** ShadowESC95

Rule of the house: if it's written here, ELI actually does it. No vapourware, no "coming soon."

What is ELI?

ELI is not a chatbot bolted onto someone else's cloud API. I had no interest in building another one of those.

It's a **local cognitive runtime**: a 12-stage reasoning pipeline, a fleet of specialist agents running on a DAG orchestrator, layered memory (SQLite + a FAISS vector index + a knowledge graph), local voice and vision, its own smart-home server, and a full desktop app *and* web dashboard — all designed to run on **your** hardware with **no required cloud service**.

You bring your own GGUF model, your own GPU stack, and — optionally — voice and model packs. ELI is model-, user-, and hardware-agnostic by design: point it at a 7B on a laptop or a 70B on a multi-GPU tower and it tunes itself to what it finds. It holds a real conversation, drives your computer, reads your screen and your documents, writes and fixes code, and quietly builds up a private picture of who you are the more you use it. Talk to it or type to it — your call.

The one promise that matters

ELI is **offline by default**, and it enforces that at the network socket — not on the honour system. Nothing leaves your machine for inference or memory. When you *do* want it to reach the internet, you flip the **Net** toggle, and even then every networked action routes through a single gate that can refuse. Your data is yours. Full stop.

The shape of the thing

Under the friendly surface, ELI is a real system. Here's the honest architecture, minus the buzzwords:

Layer	What's actually there
Reasoning	A 12-stage pipeline (intake → grounding → routing → planning → execution → review) with five selectable <i>reasoning modes</i> and confidence-driven escalation — it spends more thinking on hard problems and less on easy ones.
Agents	A fleet of specialist agents (coding, research, memory, verification, and more) coordinated by a real DAG orchestrator with parallelism, retries, fallbacks, caching and timeouts.
Memory	Four SQLite stores (user, agent, system index, coding memory) + a FAISS vector index for semantic recall + a knowledge graph for relationships. It captures salient facts on its own and promotes things you mention in passing into durable memory.
Senses	Local speech-to-text (Whisper), local text-to-speech (Piper), a local vision model for your screen and images, OCR, and optional webcam gaze control.
Hands	Full OS control — apps, windows, keyboard, mouse, media, files, shell (gated) — plus its own MQTT smart-home server for your devices.

Surfaces

A native desktop app and a web dashboard you can open from your phone on the same Wi-Fi (with a token, still 100% local).

What I believe (design principles)

- **Local or nothing.** Inference and memory never require the cloud. Offline is the default and it's enforced, not requested.
- **No fake actions.** If ELI says it'll do something, the pipeline re-runs and actually does it. No theatrical "Sure, I've done that!" while nothing happened.
- **Grounded, not guessing.** When it can't verify a fact, it says so instead of confabulating. Generation runs a real evidence step (your code, your files, the web if allowed) before it writes a word.
- **Its own voice.** ELI's persona is emergent and steered, not a script of canned lines. It adapts to you over time.
- **Honest about itself.** Ask what it's running on, how its memory works, or why it said something, and it introspects the real runtime — not a polished fiction.

Honest expectations

v2 is live software, not a shrink-wrapped product. I'm one developer, ELI touches real hardware, and there will be rough edges — especially off the Linux + NVIDIA path I run daily. It's genuinely capable and genuinely a work in progress. I'd rather have your bug report than your compliment.

What ELI Can Do

All **208 capabilities**, by category. Every row is a real action ELI routes to and executes. The example phrases are *representative* — the router is paraphrase- and typo-tolerant, so a phrase shows the *shape* that fires an action, not the only wording it accepts. Voice users just prefix with the wake word ("computer, ...").

Conversation & persona

It can...	What that means	Say something like
Hold a real conversation	Persona-bound reply; the catch-all when you're not issuing a command	"how are you", "tell me a story"
Explain its last answer	Tells you <i>why</i> it said what it said	"why did you say that"
Clear the chat	Wipes the current conversation thread	"start a new chat"
Lock / release a persona	Pin a role for the session, check it, or clear it	"lock your persona to a tutor"
Refresh its persona	Reloads its self-model from your latest profile	"refresh your persona"
List what it can do	Help + full capability enumeration	"what can you do", "help"
Run chained commands	Several actions from one sentence, in order	"close steam and set an alarm for 7am"

App & window control

It can...	What that means	Say something like
Open / close / focus apps	Launches (and will install if missing), quits, or brings an app forward	"open spotify", "close chrome"
Open the right place	Browser, a URL, the IDE, file manager, or a specific settings panel (audio, power, network, system)	"open github.com", "open audio settings"
Open the comms / media hubs	Jumps you to your mail-and-chat or media apps	"open communication hub"
Manage your windows	Tile in a grid, maximise, minimise (one / an app / everything), hide, cycle next/previous, restore	"tile windows 2x2", "show desktop"
Switch workspaces	Move between virtual desktops	"switch to workspace 2"
Control smart-home devices	Via ELI's own device server plugin	"turn off the lights"
Confirm / cancel an offer	Approve or decline a pending "install X?" / fix prompt	"yes" / "no"

Input control

It can...	What that means	Say something like
Control volume	Up, down, mute, or a specific level	"set volume to 30", "mute"
Type & press keys	Send keystrokes or type text into the active field	"press enter", "type hello world"
Move & click the mouse	Direct pointer control	"left click", "move the mouse up"

Media

It can...	What that means	Say something like
Play a specific track	On the platform you name — Spotify, or YouTube audio via headless mpv	"play Juicy by Notorious B.I.G. on Spotify"
Transport controls	Pause, stop, next, previous, repeat, shuffle, generic play/pause (MPRIS)	"skip", "shuffle", "pause the music"
Tell you what's playing	Current track and where it's coming from	"what song is this"
Skip a YouTube ad	When it's skippable	"skip the ad"

Vision & screen

It can...	What that means	Say something like
Look at your screen	Screenshot → local vision model + OCR, then answers questions about what's there	"what's on my screen", "can you see this"
Find something on screen	Locate UI text/elements	"find the submit button"
Screenshot & OCR	Capture the screen, or pull text out of an image file	"read the text in image.jpg"
Analyse images, PDFs, CSVs	Describe an image, summarise a PDF (or a whole folder of them), analyse a spreadsheet	"summarise report.pdf", "analyse data.csv"
Watch ambiently	Toggle continuous screen-watching on/off	"watch my screen", "stop watching"

Gaze (eye-tracking)

It can...	What that means	Say something like
Track your eyes via webcam	Enable, disable, calibrate, and report status	"enable gaze control", "calibrate gaze"
Click where you look	Voice-triggered clicks land where your gaze rests	"left click" / "hit enter" (gaze on)

Voice

It can...	What that means	Say something like
Dictate & transcribe	Speech into the active field; transcribe an audio file	"start dictation", "transcribe audio.wav"
Speak aloud	Local text-to-speech plugin	"say good morning"
Listen on command	Start listening; the wake word triggers it hands-free	"computer, ..."
Let you pick your wake word	Any phrase you like — it persists it and trains a local, self-supervised detector that's robust over background music	"set my wake word to atlas"
Enroll your voice	Record you saying the wake word and retrain for you specifically	"enroll my wake word"
Learn how you sound	Pitch, energy, rate, emotion, question-vs-statement — then adapts its own delivery to match	"learn how I speak"
Run voice diagnostics	Checks the STT/TTS pipeline	"run voice diagnostics"

Files & notes

It can...	What that means	Say something like
Create, read, list	Files and folders; read a file; list a directory	"create a file notes.txt", "list ~/Documents"
Audit & summarise files	Flag issues; summarise a document	"summarise report.docx"
Convert documents	Between formats	"convert report.md to pdf"
Take & manage notes	Quick notes, new notes, list, search plugin	"write a note: buy milk", "search notes for budget"
Use the clipboard	Read from / write to it	"what's in my clipboard"

Generation — documents & code

It can...	What that means	Say something like
Write grounded documents	A multi-stage pipeline — evidence → outline → sections → review → revise — not a one-shot dump	"write a report on X"
Fabricate test data	Synthetic datasets for testing	"fabricate a test dataset for X"
Write scripts & whole projects	Runnable scripts and scaffolded multi-file projects via the coding agent	"write a bash script to monitor the GPU"
Solve coding tasks	Plan → DAG → verify → repair loop	"implement function X"
Find & fix bugs	Examine a file or the whole codebase, offer fixes, apply on confirm, show diffs	"fix the bugs in foo.py", "show the diff"

System status & info

It can...	What that means	Say something like
Report the machine	CPU, RAM, GPU/VRAM, combined stats, hardware profile plugin	"gpu status", "how much ram is free"
Tell time & date	Plus a "time of a message" query and a playful sanity check	"what time is it", "when did I say that"
Weather, news, web	Weather for a place, a <i>synthesised</i> news digest, and Net-gated web search plugin	"weather in Wexford", "catch me up"

Grounded introspection — ask it about itself

It can...	What that means	Say something like
Tell you what it's running on	Live model, context length, GPU layers, batch size	"what are you running on"
Audit its own runtime	Full audit, failing imports, resolved paths, GUI wiring with file-read proof, the executor middleware chain	"run a full runtime audit"
Explain its cognition & memory	Cognition status, and a verbatim explanation of the memory runtime (all four databases)	"explain exactly how your memory works"
Show what it's aware of	Awareness snapshot, full system wiring matrix, identity provenance audit	"what are you aware of"
Explain its reasoning	Current reasoning mode and a tour of all five	"explain all your reasoning modes"
Say what it's been doing	Recent work and code changes	"what have you been working on"

Memory & profile

It can...	What that means	Say something like
Remember & recall	Store a durable fact; recall it later	"remember my sister's name is Anna"
Summarise what it knows about you	A summary, or a deep, <i>sourced</i> explanation of every fact and where it's stored	"what do you know about me"
Handle your identity	Who you are, refresh your profile, set/correct your name	"call me Alex", "who am I"

Self-maintenance — it looks after itself

It can...	What that means	Say something like
Analyse & improve itself	Study its own failures, run a patch cycle, keep a self-improvement log	"analyse your failures", "improve yourself"
Upgrade itself	git pull → deps → rebuild indexes	"upgrade yourself"
Test itself	Self-tests, the full pytest suite with a summarised report, and a review that offers result-driven fixes	"run the tests and tell me what to fix"
Write its own tests	Generates & sandbox-verifies behavioural tests for its own functions	"grow your test coverage"
Fine-tune itself (LoRA)	Preflight → build → train → eval DAG; dry-run from chat, real training overnight	"is lora ready", "fine-tune yourself"
Explain its orchestrator	The agent DAG: layers, dependencies, critical path, last run	"show your agent dag"

Tasks, time & planning

It can...	What that means	Say something like
Do work overnight	Schedule timed/overnight code, research, evals — durable across restarts	"research X overnight", "build Y at 2am"
Alarms, timers, events	Set alarms and timers; add and list calendar events	"set a timer for 10 minutes"
Pomodoro & jobs	Start/stop/status a pomodoro plugin ; list and check background jobs	"start a pomodoro", "show background jobs"
Learn & propose habits	Surfaces habits it has noticed; you approve or decline before it acts	"show my habits"
Execute goals	Run a stored goal	"execute goal X"

Proactive — it comes to you

It can...	What that means	Say something like
Run a proactive daemon	Start, stop, status — background awareness that watches for useful moments	<i>"start proactive mode"</i>
Propose things	Self-generated proposals and goals from signals it has observed	<i>"what's on your agenda"</i>
Brief you each morning	News, weather, agenda in one digest	<i>"give me my briefing"</i>

Plugins & shell

It can...	What that means	Say something like
Manage plugins	List, search, install, uninstall, enable, disable	<i>"install the X plugin"</i>
Run shell commands	Gated and confirmed — nothing runs silently	<i>"run the command: ls -la"</i>
Run sequences	Multi-step action chains	<i>"do X then Y then Z"</i>
Explain a routing miss	Tells you why an input didn't map to an action	<i>"why did that fail to route"</i>

...and the part that isn't a command

Beyond these 183 directly-triggerable actions sit dozens of internal ones — synonym normalisation, post-actions, pipeline steps, safety guards — plus the whole conversational, reasoning, and memory substrate that makes the commands feel like talking to something coherent rather than barking at a menu. The number on the box is 208. The thing you actually feel is *one assistant that gets you*.

ELI v2.0 — a strictly local, private AI assistant and cognitive runtime. Built by one person, run on your machine, owned by you. Full command syntax lives in **Commands & Installers**; step-by-step usage lives in the **User Manual**.